

AIMLCZG546 · SESSION 2 · COMPANION READER

Machine Learning in Production

From Research Prototype to Production System: Scope, Challenges, and Architecture

Session 2 · Read alongside the lecture slides · Every concept and case study explained in full

How to use this companion. Five colour-coded boxes appear throughout. **Blue** = core intuition. **Amber** = everyday analogy. **Green** = worked scenario. **Red** = pitfall. **Purple** = the one line to remember.

1 The Production Gap: Sidney's Startup

A common story in ML: a talented researcher builds an impressive model in a notebook. Accuracy is high on the test set. Everyone is excited. Then they try to deploy it. That is when the real work begins.

★ Everyday picture

A chef perfects a recipe at home for six people. Then a restaurant chain wants to serve it to 10,000 people per day across 50 cities. The recipe itself is only a tiny part of the problem. The real challenges are: sourcing consistent ingredients, training kitchen staff, maintaining quality control, handling customer complaints, and managing costs.

Sidney builds a speech recognition model for customer calls. In the lab, accuracy is 94%. In production, he encounters:

- **Noisy real-world data** — call centre audio has background noise, accents, and technical jargon not in the training set.
- **Latency constraints** — the model must respond in under 500 ms; the research model takes 3 seconds.
- **Cloud cost explosion** — running the model 24/7 at scale costs 40× the research estimate.
- **Pipeline fragility** — the data preprocessing script breaks when the call centre upgrades its recording format.
- **Team gaps** — ML researchers and software engineers don't share vocabulary or tools.
- **Monitoring failures** — accuracy degrades after six months (customers' language changed) but nobody notices for three weeks.
- **Fairness gaps** — the model performs 15% worse on non-native English speakers.

✓ Key takeaway

Moving from research prototype to production system requires as much engineering effort as building the model itself — often more.

2 ML Model vs ML System

There are two very different ways to think about an ML project.

2.1 Model-Centric View

A **model-centric** team asks: “How do we make the model more accurate?” They tune hyperparameters, try new architectures, and collect more labelled data. Improvement stops when the model tops out.

2.2 System-Centric View

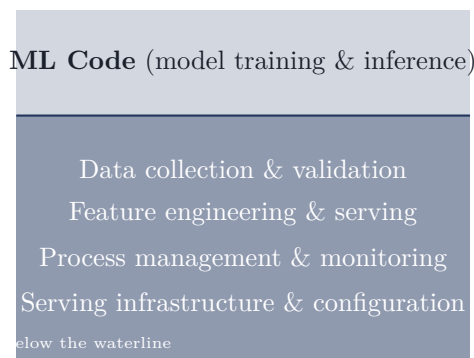
A **system-centric** team asks: “How do we make the entire end-to-end system better for users?” The model is one component of a larger system. Improvements can come from better data pipelines, faster infrastructure, better monitoring, or better UX — not just the model.

The intuition

An ML model is like an engine in a car. A better engine helps, but the car’s usefulness also depends on the chassis, the steering, the tyres, the fuel system, and the driver interface. Optimising only the engine while ignoring everything else is model-centric thinking.

2.3 The Iceberg of Hidden Technical Debt

Sculley et al. (2015) — a landmark paper from Google — showed that the actual ML code in a production system is a tiny fraction of the total codebase. The rest is the infrastructure:

**✗ Watch out**

Optimising the ML model while neglecting the infrastructure is like polishing the engine while the fuel tank leaks. The fuel tank will kill the car first.

Worked example: Model-centric vs system-centric fraud detection

A bank’s fraud detection model has 88 % precision (12 % false positives — legitimate transactions blocked).

Model-centric fix: Collect more labelled fraud examples. Retrain with XGBoost. New precision: 90 %. Cost: 3 weeks.

System-centric fix: Analyse why false positives occur. Discovery: the feature pipeline is using yesterday's exchange rates for international transactions. Fix the pipeline. New precision: 93 %, with no model change at all. Cost: 2 days.

The system-centric approach found a bigger gain faster by looking beyond the model.

3 ML and Non-ML Components in a System

Real production systems mix ML and traditional software. The ML component may be central (**core**) or supporting (**auxiliary**).

System	ML role	Non-ML components
Speech transcription service	Core (the product IS the ML output)	Audio codec, API gateway, billing
Tax software audit risk	Auxiliary (flags for human review)	Form parsing, rule engine, UI, tax database
E-commerce recommendation	Core (drives revenue directly)	Product catalogue, cart, payment, logistics

Where this shows up in ML/AI. Systems thinking means you must design every non-ML component around the ML component's failure modes. When the fraud model makes a mistake, who sees it, what happens next, and how do you recover? These are software engineering questions, not ML questions.

4 AI Paradigms: Predictive, Generative, and Agentic

Modern AI systems fall into three paradigms, each with different capabilities and engineering requirements.

4.1 Predictive AI

Predictive AI uses past data to predict future outcomes. The model is trained on historical data and answers a specific question ("Will this customer churn?", "Is this email spam?"). Output is a number, a label, or a probability.

4.2 Generative AI

Generative AI creates new content based on patterns learned from data. A foundation model is given a natural-language instruction and produces a digital artefact: text, code, an image, a video.

The intuition

Predictive AI is like a detective who studies evidence and says "the suspect is probably the butler." Generative AI is like a novelist who reads a thousand mystery novels and then writes a new one.

4.3 Agentic AI

Agentic AI acts autonomously to achieve a goal. It combines reasoning (planning multi-step actions), tools (calling APIs, searching the web, running code), and feedback loops (evaluating its own output and correcting errors).

$$\text{Agentic AI} = \text{Generative AI} + \text{Reasoning} + \text{Tools} + \text{Feedback Loops}$$

★ Everyday picture

Predictive AI is a weather forecast app: you ask, it answers. Generative AI is a ghostwriter who drafts the email you asked for. Agentic AI is a personal assistant who reads your brief, books the flight, writes the agenda, sends the meeting invite, and then revises the agenda when you tell them the venue changed.

✓ Key takeaway

Agentic AI systems are not just smarter chatbots — they are autonomous software agents that can take sequences of real-world actions. This changes the SE challenges significantly (coordination, safety, rollback).

5 Cloud-Native ML Systems

Cloud-native software is designed from the start to run on cloud infrastructure using containers, microservices, and automated deployment. The Cloud Native Computing Foundation (CNCF) defines it as: technologies that use containers, service meshes, microservices, immutable infrastructure, and declarative APIs.

For ML systems, cloud-native means:

- Models run in containers (reproducible, isolated environments).
- Training and serving scale independently (more GPUs for training; more API servers for serving).
- Infrastructure is defined as code (Terraform, Kubernetes YAML) so it is versioned and auditable.
- ML workloads are the dominant and fastest-growing type of cloud compute today.

6 Case Study: Apollo Autonomous Driving (Baidu)

Baidu's Apollo platform is one of the most complex ML systems ever deployed in a safety-critical context. It illustrates what “ML system” means at industrial scale.

Apollo uses **28 ML models** across three functional areas:

- **Perception** — cameras and LiDAR sensors feed models that detect lanes, vehicles, pedestrians, traffic signs, and road boundaries.
- **Detection & Classification** — models classify objects and predict their type (car, cyclist, pedestrian).
- **Trajectory Prediction** — models forecast where detected objects will be in the next 3–5 seconds.

Worked example: Model interaction in Apollo

The pipeline for a single frame of camera data:

Step 1: Raw camera image (1920×1080, 30 fps) is received.

Step 2: Object Detection model (YOLOv5 variant) runs. Output: 14 bounding boxes

(cars, pedestrians, cyclists).

Step 3: Each bounding box is fed to the Classification model. Output: “Car (0.97), Pedestrian (0.91), Truck (0.83)...”

Step 4: Classification outputs + LiDAR point cloud are fused by the Multi-Sensor Fusion model. Output: 3D positions of all objects.

Step 5: Trajectory Prediction model takes 3D positions + velocity history. Output: predicted paths for next 4 seconds.

Step 6: Motion Planning (rule-based + ML) uses predicted paths to compute a safe driving trajectory.

Step 7: Control system executes the trajectory (steering, throttle, brake).

If Step 2 produces a false negative (misses a pedestrian), every downstream step receives wrong input. This cascading failure risk is the core SE challenge of multi-model systems.

SE4ML challenges in Apollo: high system complexity (28 interacting models), very low test coverage (hard to write unit tests for probabilistic models), model maintenance (each of 28 models must be versioned and retrained separately), and safety requirements (a wrong prediction may kill someone).

7 Case Study: Microsoft’s 9-Stage ML Workflow

Microsoft studied how ML is embedded in Agile engineering teams and found a consistent 9-stage workflow.

Stage	Name	Key question
1	Model Requirements	What problem are we solving? What does success look like?
2	Data Collection	Where does the data come from? Is it available?
3	Data Cleaning	Is the data accurate, complete, and consistent?
4	Data Labelling	Do we have ground truth? Who labels it?
5	Feature Engineering	What signals will help the model?
6	Model Training	What algorithm? What hyperparameters?
7	Model Evaluation	Does it meet the performance requirements?
8	Deployment	How is the model served? What latency / throughput?
9	Monitoring	Is the model still performing well in production?

Key findings from Microsoft’s research:

- **Data is the central engineering activity.** More time is spent on stages 2–5 than on 6–7.
- **Experimentation-driven development.** Teams run dozens of experiments before choosing a model.
- **Cross-functional teams are essential.** ML engineers, data engineers, domain experts, and product managers must collaborate from stage 1.
- **Versioning is not optional.** Both models and datasets must be versioned to ensure reproducibility.

✓ Key takeaway

In the Microsoft workflow, the model training stage (Stage 6) is preceded by four data-intensive stages. “Data first, model second” is the industrial reality.